

INDEX

S.N	Title	Page No.	Signature
1	Write a C Program to Implement DDA Line Drawing Algorithm.	1	
2	Write a C Program to Implement Bresenham's Line Drawing Algorithm.	3	
3	Write a C Program to Implement Midpoint circle Algorithm.	6	
4	Write a C Program to Implement Midpoint Ellipse Algorithm.	8	
5	Write a C Program to Implement Translation of a Triangle.	11	
6	Write a C Program to Implement Rotation of a Triangle.	13	
7	Write a C Program to Implement Scaling of a Triangle.	15	
8	Write a C Program to Implement Reflection of a Triangle.	17	
9	Write a C Program to Implement Shearing of a Triangle.	19	
10	Write a C Program to Implement Line clipping algorithm.	21	

Signature:.....

1. Write a C Program to Implement DDA Line Drawing Algorithm.

Code:

```
#include <stdio.h>

#include <conio.h>

#include <graphics.h>

#include <math.h>

void main()

{

    int gd = DETECT, gm;

    int x1, y1, x2, y2, stepsize, dx, dy, i;

    float x, y, xinc, yinc;

    initgraph(&gd, &gm, "c:\\tc\\bgi");

    printf("Enter x1 and y1: ");

    scanf("%d %d", &x1, &y1);

    printf("Enter x2 and y2: ");

    scanf("%d %d", &x2, &y2);

    dx = x2 - x1;

    dy = y2 - y1;

    if (abs(dy) > abs(dx))

        stepsize = abs(dy);

    else

        stepsize = abs(dx);

    xinc = dx / (float)stepsize;

    yinc = dy / (float)stepsize;

    x = x1;

    y = y1;

    putpixel(x, y, RED);

    for (i = 0; i < stepsize; i++)

    {
```

```

    x = x + xinc;

    y = y + yinc;

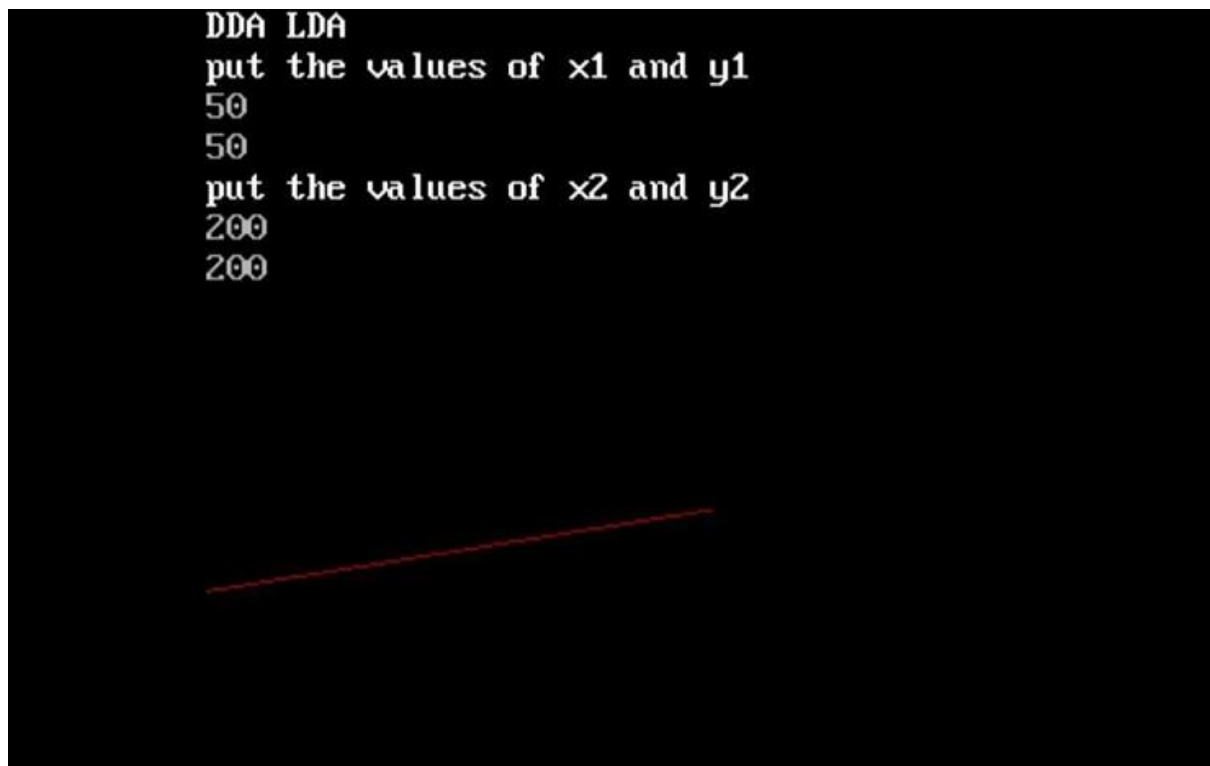
    putpixel((int)(x + 0.5), (int)(y + 0.5), RED);
}

getch();

closegraph();
}

```

Output:



2. Write a C Program to Implement Bresenham's Line Drawing Algorithm.

Code:

```
#include <stdio.h>

#include <conio.h>

#include <graphics.h>

#include <math.h>

void main()

{

    int gd = DETECT, gm;

    int x1, y1, x2, y2, dx, dy, lx, ly, pk, i;

    initgraph(&gd, &gm, "c:\\tc\\bgi");

    printf("Enter x1 and y1: ");

    scanf("%d %d", &x1, &y1);

    printf("Enter x2 and y2: ");

    scanf("%d %d", &x2, &y2);

    dx = abs(x2 - x1);

    dy = abs(y2 - y1);

    if (x2 > x1)

        lx = 1;

    else

        lx = -1;

    if (y2 > y1)

        ly = 1;

    else

        ly = -1;

    putpixel(x1, y1, RED);

    if (dx > dy)

    {

        pk = 2 * dy - dx;
```

```

for (i = 0; i < dx; i++)
{
    if (pk < 0)
    {
        x1 = x1 + lx;
        pk = pk + 2 * dy;
    }
    else
    {
        x1 = x1 + lx;
        y1 = y1 + ly;
        pk = pk + 2 * dy - 2 * dx;
    }
    putpixel(x1, y1, RED);
}
else
{
    pk = 2 * dx - dy;
    for (i = 0; i < dy; i++)
    {
        if (pk < 0)
        {
            y1 = y1 + ly;
            pk = pk + 2 * dx;
        }
        else
        {
            x1 = x1 + lx;

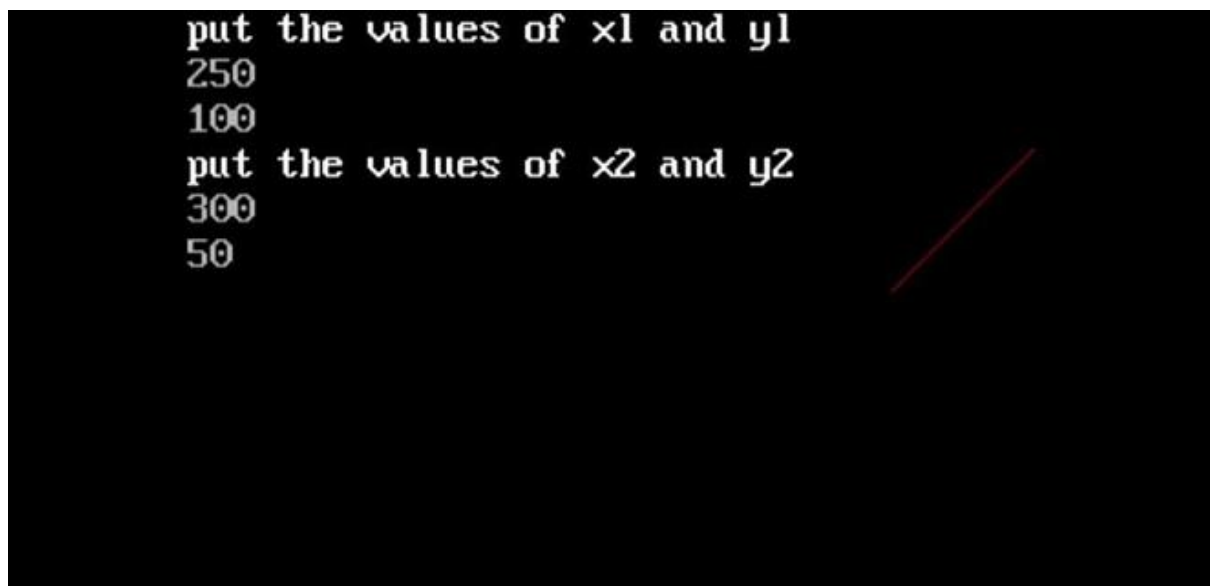
```

```

        y1 = y1 + ly;
        pk = pk + 2 * dx - 2 * dy;
    }
    putpixel(x1, y1, RED);
}
}
getch();
closegraph();
}

```

Output:



3. Write a C Program to Implement Midpoint circle Algorithm.

Code:

```
#include <conio.h>

#include <graphics.h>

#include <stdio.h>

void main()

{

    int x, y, xc, yc, r, d;

    int gd = DETECT, gm;

    initgraph(&gd, &gm, "C:\\\\TURBOC3\\\\BGI");

    printf("* MID POINT CIRCLE DRAWING ALGORITHM *\n");

    printf("Enter center coordinates (xc yc): ");

    scanf("%d %d", &xc, &yc);

    printf("Enter radius: ");

    scanf("%d", &r);

    x = 0;

    y = r;

    d = 1 - r;

    do

    {

        putpixel(xc + x, yc + y, 1);

        putpixel(xc + y, yc + x, 1);

        putpixel(xc - y, yc + x, 1);

        putpixel(xc - x, yc + y, 1);

        putpixel(xc - x, yc - y, 1);

        putpixel(xc - y, yc - x, 1);

        putpixel(xc + y, yc - x, 1);

        putpixel(xc + x, yc - y, 1);

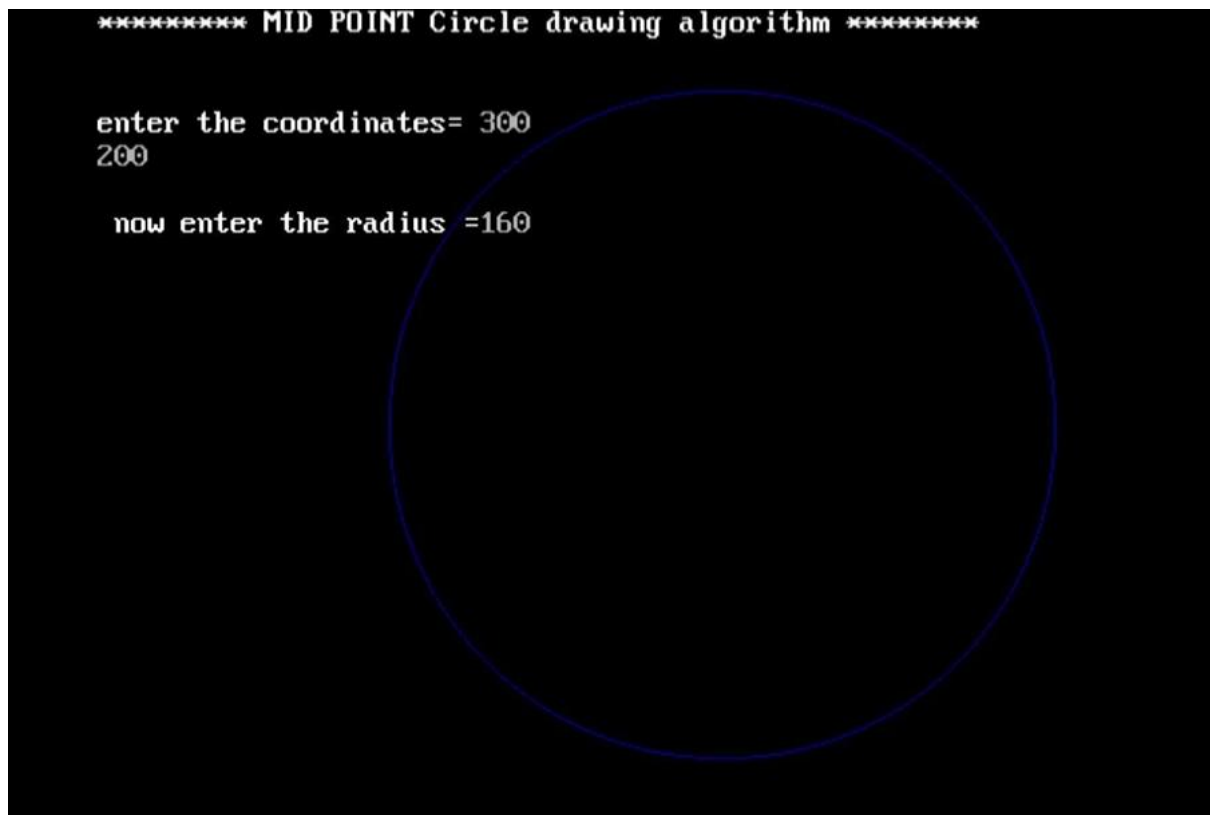
        if (d < 0)
```

```

{
    d = d + (2 * x) + 3;
}
else
{
    d = d + (2 * (x - y)) + 5;
    y--;
}
x++;
} while (x <= y);
getch();
closegraph();
}

```

Output:



4. Write a C Program to Implement Midpoint Ellipse Algorithm.

Code:

```
#include <stdio.h>

#include <graphics.h>

#include <conio.h>

void main()

{

    long x, y, xc, yc;

    long a, b;

    long a_sqr, b_sqr;

    long fx, fy;

    float d, tmp1, tmp2;

    int gd = DETECT, gm;

    initgraph(&gd, &gm, "C:\\\\TURBOC3\\\\BGI");

    printf("* MID POINT ELLIPSE ALGORITHM *\n");

    printf("Enter center coordinates (xc yc): ");

    scanf("%ld %ld", &xc, &yc);

    printf("Enter a and b: ");

    scanf("%ld %ld", &a, &b);

    x = 0;

    y = b;

    a_sqr = a * a;

    b_sqr = b * b;

    fx = 2 * b_sqr * x;

    fy = 2 * a_sqr * y;

    d = b_sqr - (a_sqr * b) + (0.25 * a_sqr);

    /* Region 1 */

    while (fx < fy)

    {
```

```

    putpixel(xc + x, yc + y, 1);
    putpixel(xc - x, yc + y, 1);
    putpixel(xc + x, yc - y, 1);
    putpixel(xc - x, yc - y, 1);
    if (d < 0)
    {
        d = d + fx + b_sqr;
    }
    else
    {
        y--;
        fy = fy - (2 * a_sqr);
        d = d + fx - fy + b_sqr;
    }
    x++;
    fx = fx + (2 * b_sqr);
}
/* Region 2 */
tmp1 = (x + 0.5) * (x + 0.5);
tmp2 = (y - 1) * (y - 1);
d = (b_sqr * tmp1) + (a_sqr * tmp2) - (a_sqr * b_sqr);
while (y > 0)
{
    putpixel(xc + x, yc + y, 1);
    putpixel(xc - x, yc + y, 1);
    putpixel(xc + x, yc - y, 1);
    putpixel(xc - x, yc - y, 1);
    if (d > 0)
    {

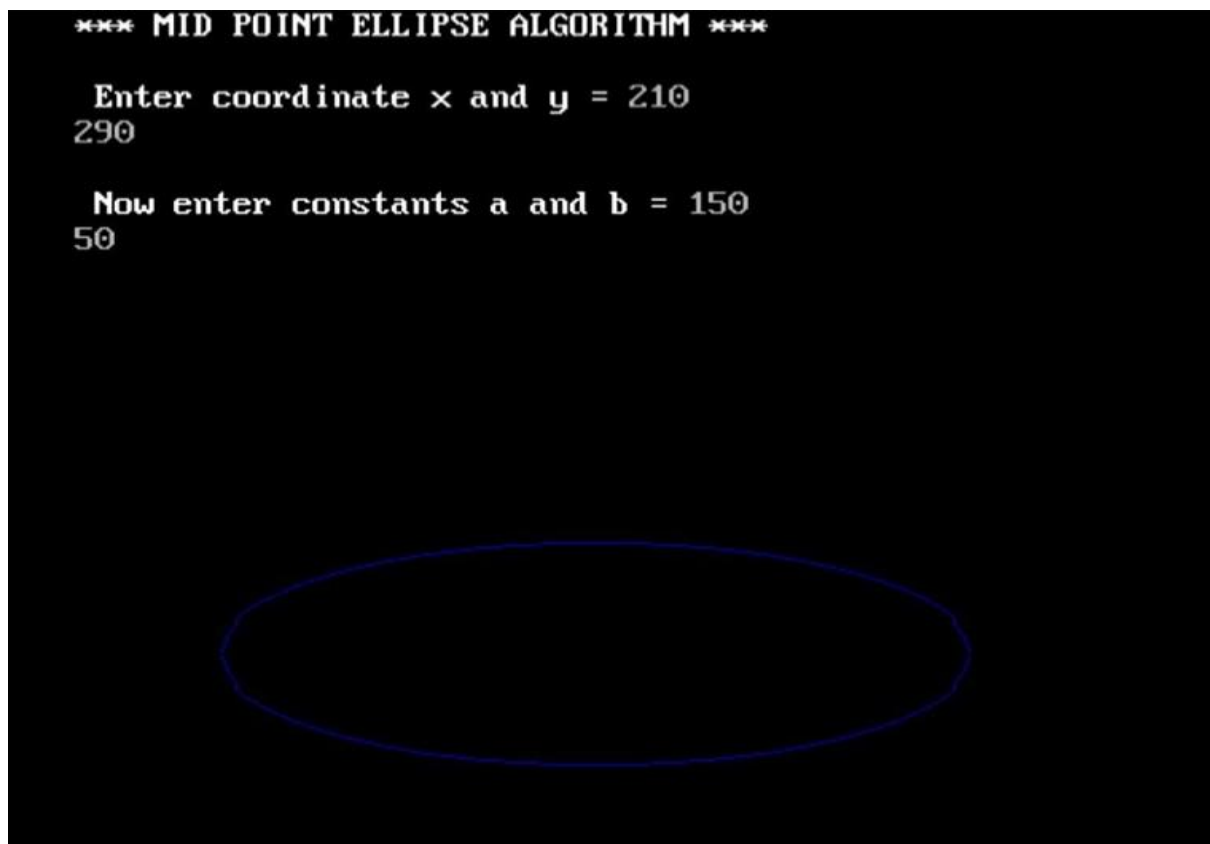
```

```

        d = d - fy + a_sqr;
    }
    else
    {
        x++;
        fx = fx + (2 * b_sqr);
        d = d + fx - fy + a_sqr;
    }
    y--;
    fy = fy - (2 * a_sqr);
}
getch();
closegraph();
}

```

Output:



5. Write a C Program to Implement Translation of a Triangle.

Code:

```
#include<conio.h>

#include<graphics.h>

#include<stdio.h>

void main()

{

    int gd=DETECT, gm;

    int x,y,x1,y1,x2,y2,tx,ty;

    clrscr();

    initgraph(&gd,&gm,"C:\\TURBOC3\\BGI");

    printf("\n Please enter first coordinate of the triangle= ");

    scanf("%d %d",&x,&y);

    printf("\n Enter second coordinate of the triangle = ");

    scanf("%d %d",&x1,&y1);

    printf("\n Enter third coordinate of the triangle = ");

    scanf("%d %d",&x2,&y2);

    printf("\n\t***** TRIANGLE before & after translation *****");

    line(x,y,x1,y1);

    line(x1,y1,x2,y2);

    line(x2,y2,x,y);

    printf("\n Now enter the translation vector = ");

    scanf("%d %d",&tx,&ty);

    setcolor(RED);

    line(x+tx,y+ty,x1+tx,y1+ty);

    line(x1+tx,y1+ty,x2+tx,y2+ty);

    line(x2+tx,y2+ty,x+tx,y+ty);

    getch();

    closegraph();
```

}

Output:

```
Please enter first coordinate of the triangle= 40
100
Enter second coordinate of the triangle = 300
60
Enter third coordinate of the triangle = 100
300
***** TRIANGLE before & after translation *****
Now enter the translation vector = 50
-100
```

6. Write a C Program to Implement Rotation of a Triangle.

Code:

```
#include <stdio.h>

#include <conio.h>

#include <graphics.h>

#include <math.h>

void DrawTriangle(int x1, int y1, int x2, int y2, int x3, int y3);

void RotateTriangle(int x1, int y1, int x2, int y2, int x3, int y3, float angle);

int main()

{

    int gd = DETECT, gm;

    int x1, y1, x2, y2, x3, y3;

    float angle;

    initgraph(&gd, &gm, "C:\\\\TURBOC3\\\\BGI");

    printf("Enter the 1st point for the triangle (x1 y1): ");

    scanf("%d%d", &x1, &y1);

    printf("Enter the 2nd point for the triangle (x2 y2): ");

    scanf("%d%d", &x2, &y2);

    printf("Enter the 3rd point for the triangle (x3 y3): ");

    scanf("%d%d", &x3, &y3);

    DrawTriangle(x1, y1, x2, y2, x3, y3);

    printf("Enter the angle for rotation (in degrees): ");

    scanf("%f", &angle);

    RotateTriangle(x1, y1, x2, y2, x3, y3, angle);

    getch();

    closegraph();

    return 0;

}

void DrawTriangle(int x1, int y1, int x2, int y2, int x3, int y3)
```

```

{
    line(x1, y1, x2, y2);
    line(x2, y2, x3, y3);
    line(x3, y3, x1, y1);
}

void RotateTriangle(int x1, int y1, int x2, int y2, int x3, int y3, float angle)
{
    int p = x2, q = y2;
    float radianAngle = (angle * 3.14) / 180.0;
    int a1 = p + (x1 - p) * cos(radianAngle) - (y1 - q) * sin(radianAngle);
    int b1 = q + (x1 - p) * sin(radianAngle) + (y1 - q) * cos(radianAngle);
    int a2 = p + (x2 - p) * cos(radianAngle) - (y2 - q) * sin(radianAngle);
    int b2 = q + (x2 - p) * sin(radianAngle) + (y2 - q) * cos(radianAngle);
    int a3 = p + (x3 - p) * cos(radianAngle) - (y3 - q) * sin(radianAngle);
    int b3 = q + (x3 - p) * sin(radianAngle) + (y3 - q) * cos(radianAngle);
    setcolor(1);
    DrawTriangle(a1, b1, a2, b2, a3, b3);
}

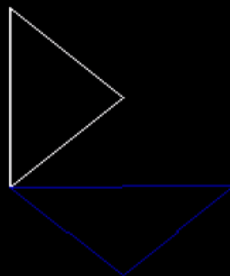
```

Output:

```

Enter the 1st point for the triangle (x1 y1): 200 200
Enter the 2nd point for the triangle (x2 y2): 200 300
Enter the 3rd point for the triangle (x3 y3): 250 250
Enter the angle for rotation (in degrees): 90

```



7. Write a C Program to Implement Scaling of a Triangle.

Code:

```
#include<stdio.h>

#include<conio.h>

#include<graphics.h>

void main()

{

    int x,y,x1,y1,x2,y2;

    int scl_fctr_x,scl_fctr_y;

    int gd=DETECT,gm;

    initgraph(&gd,&gm,"C:\\TURBOC3\\BGI");

    printf("\t\t** Scaling **\n");

    printf("\n Please enter first coordinate of Triangle = ");

    scanf("%d %d",&x,&y);

    printf("\n Please enter second coordinate of Triangle = ");

    scanf("%d %d",&x1,&y1);

    printf("\n Please enter third coordinate of Triangle = ");

    scanf("%d %d",&x2,&y2);

    line(x,y,x1,y1);

    line(x1,y1,x2,y2);

    line(x2,y2,x,y);

    printf("\n Now Enter Scaling factor x and y = ");

    scanf("%d %d",&scl_fctr_x,&scl_fctr_y);

    x = x * scl_fctr_x;

    x1 = x1 * scl_fctr_x;

    x2 = x2 * scl_fctr_x;

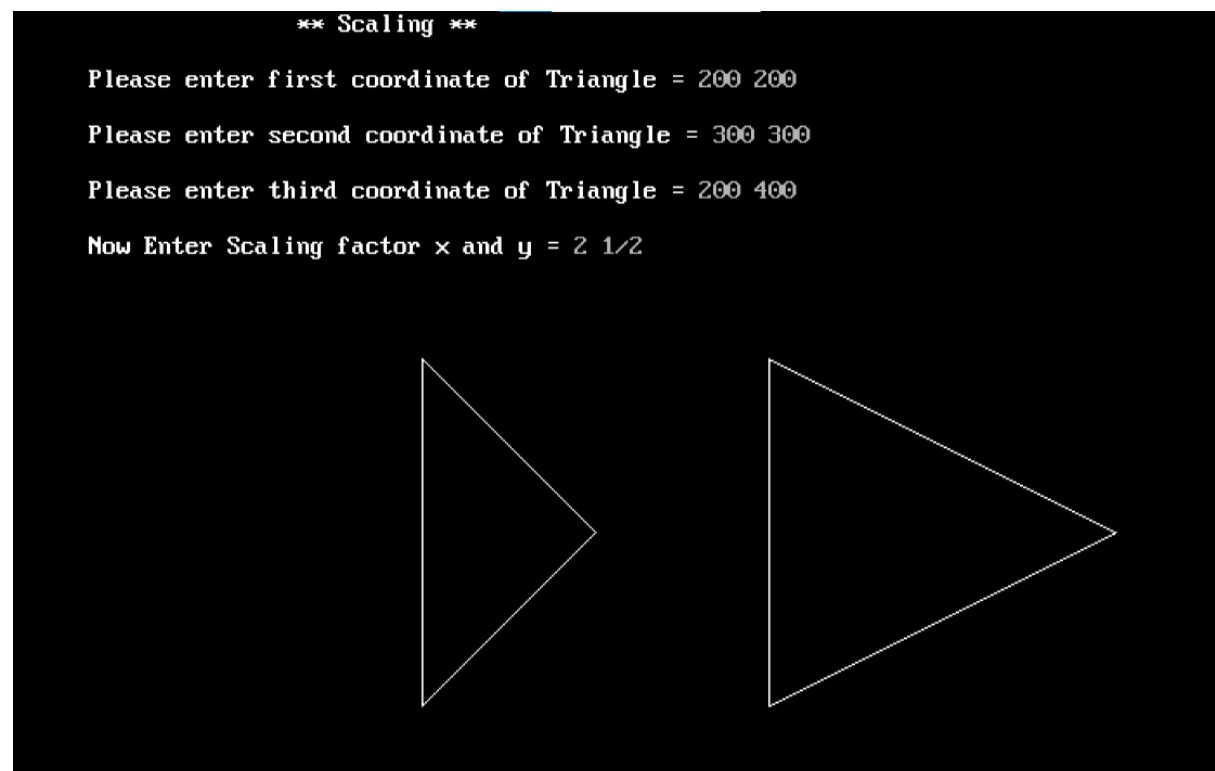
    y = y * scl_fctr_y;

    y1 = y1 * scl_fctr_y;

    y2 = y2 * scl_fctr_y;
```

```
line(x,y,x1,y1);  
line(x1,y1,x2,y2);  
line(x2,y2,x,y);  
getch();  
closegraph();  
}
```

Output:



8. Write a C Program to Implement Reflection of a Triangle.

Code:

```
#include <conio.h>

#include <graphics.h>

#include <stdio.h>

void main()

{

    int gm, gd = DETECT, ax, x1 = 100;

    int x2 = 100, x3 = 200, y1 = 100;

    int y2 = 200, y3 = 100;

    initgraph(&gd,&gm,"C:\\\\TURBOC3\\\\BGI");

    cleardevice();

    line(getmaxx()/2,0,getmaxx()/2,getmaxy());

    line(0,getmaxy()/2,getmaxx(),getmaxy()/2);

    printf("Before Reflection Object in 2nd Quadrant");

    setcolor(14);

    line(x1,y1,x2,y2);

    line(x2,y2,x3,y3);

    line(x3,y3,x1,y1);

    getch();

    printf("\nAfter Reflection");

    setcolor(4);

    line(getmaxx()-x1,getmaxy()-y1,getmaxx()-x2,getmaxy()-y2);

    line(getmaxx()-x2,getmaxy()-y2,getmaxx()-x3,getmaxy()-y3);

    line(getmaxx()-x3,getmaxy()-y3,getmaxx()-x1,getmaxy()-y1);

    setcolor(3);

    line(getmaxx()-x1,y1,getmaxx()-x2,y2);

    line(getmaxx()-x2,y2,getmaxx()-x3,y3);

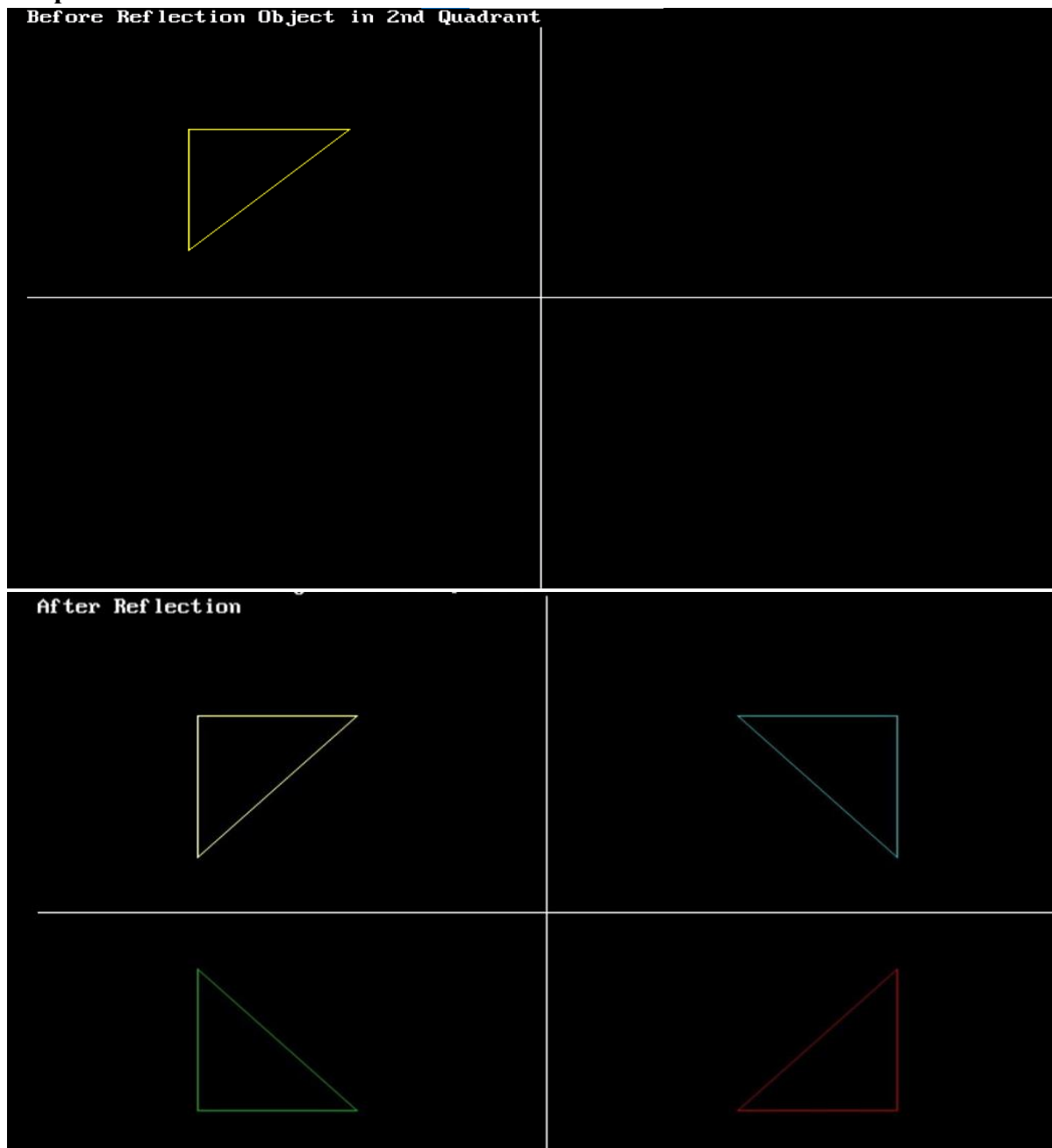
    line(getmaxx()-x3,y3,getmaxx()-x1,y1);
```

```

setcolor(2);
line(x1,getmaxy()-y1,x2,getmaxy()-y2);
line(x2,getmaxy()-y2,x3,getmaxy()-y3);
line(x3,getmaxy()-y3,x1,getmaxy()-y1);
getch();
closegraph();
}

```

Output:



9. Write a C Program to Implement Shearing of a Triangle.

Code:

```
#include<stdio.h>

#include<graphics.h>

#include<conio.h>

void main()

{

    int gd=DETECT,gm;

    int x,y,x1,y1,x2,y2,shear_f;

    initgraph(&gd,&gm,"C:\\TURBOC3\\BGI");

    printf("\n please enter first coordinate = ");

    scanf("%d %d",&x,&y);

    printf("\n please enter second coordinate = ");

    scanf("%d %d",&x1,&y1);

    printf("\n please enter third coordinate = ");

    scanf("%d %d",&x2,&y2);

    printf("\n please enter shearing factor x = ");

    scanf("%d",&shear_f);

    cleardevice();

    line(x,y,x1,y1);

    line(x1,y1,x2,y2);

    line(x2,y2,x,y);

    setcolor(RED);

    x = x + y*shear_f;

    x1 = x1 + y1*shear_f;

    x2 = x2 + y2*shear_f;

    line(x,y,x1,y1);

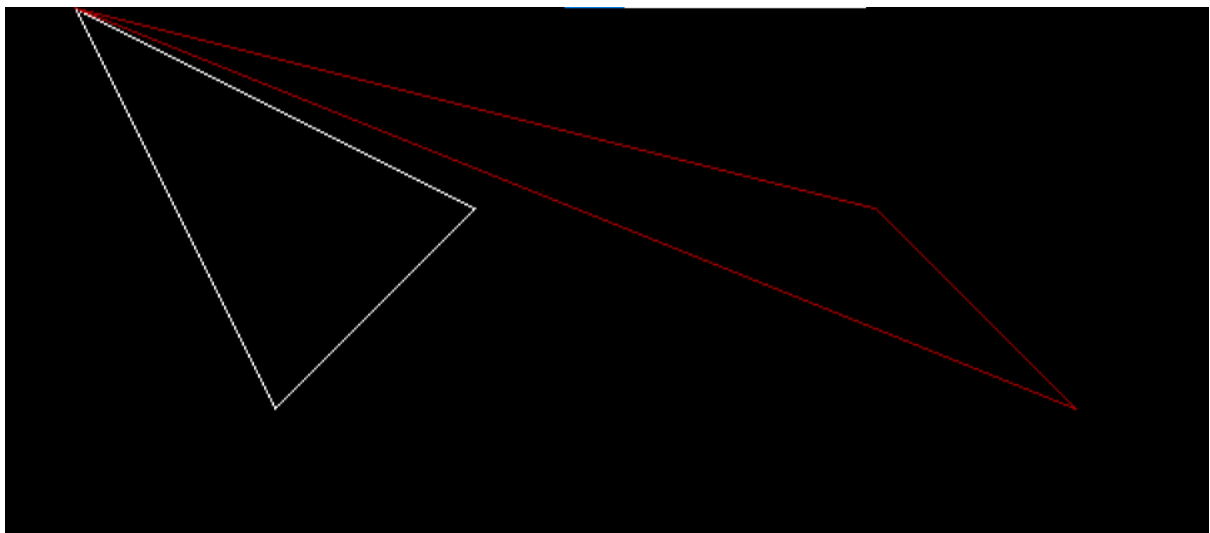
    line(x1,y1,x2,y2);

    line(x2,y2,x,y);
```

```
getch();  
closegraph();  
}
```

Output:

```
please enter first coordinate = 0  
0  
  
please enter second coordinate = 100  
200  
  
please enter third coordinate = 200  
100  
  
please enter shearing factor x = 2
```



10. Write a C Program to Implement Line clipping algorithm.

Code:

```
#include <graphics.h>
#include <stdio.h>

#define INSIDE 0
#define LEFT 1
#define RIGHT 2
#define BOTTOM 4
#define TOP 8
#define X_MIN 50
#define Y_MIN 50
#define X_MAX 250
#define Y_MAX 250

int computeRegionCode(int x, int y)
{
    int code = INSIDE;
    if (x < X_MIN)
        code |= LEFT;
    else if (x > X_MAX)
        code |= RIGHT;
    if (y < Y_MIN)
        code |= BOTTOM;
    else if (y > Y_MAX)
        code |= TOP;
    return code;
}

void cohenSutherland(int x1, int y1, int x2, int y2)
{
    int code1 = computeRegionCode(x1, y1);
```

```

int code2 = computeRegionCode(x2, y2);
int accept = 0;
while (1)
{
    if (code1 == 0 && code2 == 0)
    {
        accept = 1;
        break;
    }
    else if (code1 & code2)
    {
        break;
    }
    else
    {
        int codeOut = code1 ? code1 : code2;
        int x, y;

        if (codeOut & TOP)
        {
            x = x1 + (x2 - x1) * (Y_MAX - y1) / (y2 - y1);
            y = Y_MAX;
        }
        else if (codeOut & BOTTOM)
        {
            x = x1 + (x2 - x1) * (Y_MIN - y1) / (y2 - y1);
            y = Y_MIN;
        }
        else if (codeOut & RIGHT)

```

```

    {
        y = y1 + (y2 - y1) * (X_MAX - x1) / (x2 - x1);
        x = X_MAX;
    }
    else if (codeOut & LEFT)
    {
        y = y1 + (y2 - y1) * (X_MIN - x1) / (x2 - x1);
        x = X_MIN;
    }
    if (codeOut == code1)
    {
        x1 = x;
        y1 = y;
        code1 = computeRegionCode(x1, y1);
    }
    else
    {
        x2 = x;
        y2 = y;
        code2 = computeRegionCode(x2, y2);
    }
}

if (accept)
{
    setcolor(RED);
    line(x1, y1, x2, y2);
}
}

```

```

int main()
{
    int gd = DETECT, gm;
    int x1, y1, x2, y2;
    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");
    printf("Enter coordinates of the line (x1 y1 x2 y2): ");
    scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
    rectangle(X_MIN, Y_MIN, X_MAX, Y_MAX);
    setcolor(BLUE);
    line(x1, y1, x2, y2);
    cohenSutherland(x1, y1, x2, y2);
    getch();
    closegraph();
return 0;
}

```

Output:

